

Python pour le trading — Document 10

Les environnements

1. Qu'est-ce qu'un environnement, et pourquoi s'en soucier

Un projet Python ne tourne presque jamais seul. Il dépend d'une **version de Python** précise et d'un ensemble de **paquets externes** (NumPy, pandas, requests...) eux aussi à des versions précises. Cette combinaison — une version de Python et ses paquets — est ce qu'on appelle un **environnement**. C'est le contexte dans lequel ton code s'exécute.

Un environnement répond à deux besoins. Le premier est l'**isolation** : les paquets d'un projet ne doivent pas perturber ceux d'un autre. Le second est la **reproductibilité** : pouvoir recréer exactement le même contexte ailleurs — sur la machine d'un collègue, sur un serveur de production, ou sur la tienne dans six mois.

Sans environnements, tout s'installe au même endroit partagé : les projets finissent par se gêner, et les résultats deviennent impossibles à reproduire à l'identique. Ce module part de la notion centrale qui prête le plus à confusion — l'**interpréteur** — puis montre le problème concret, les **méthodes** pour créer des environnements, et comment elles se comparent.

***Pourquoi ça compte.** En trading, un même code doit produire le même résultat du poste de développement au serveur de production. L'environnement est précisément l'outil qui garantit cette identité ; le négliger, c'est s'exposer à des écarts inexpliqués entre « ça marche chez moi » et la réalité en production.*

2. L'interpréteur : la notion qui embrouille

Un **interpréteur** est simplement un fichier exécutable python (par exemple `C:\...\projet\venv\Scripts\python.exe`). Un environnement virtuel, ce n'est rien d'autre qu'un dossier contenant un tel `python.exe` accompagné de ses propres paquets. « Utiliser un environnement », c'est faire en sorte qu'un programme appelle CE `python.exe`-là plutôt qu'un autre.

Or « choisir l'interpréteur » se fait à **deux endroits totalement indépendants** — et c'est la source de la confusion la plus fréquente.

1. Dans le terminal. La commande d'activation (`activate`) modifie la fenêtre de terminal pour que, dans CETTE fenêtre uniquement, `python` et `pip` désignent le `python.exe` de l'environnement. L'effet est limité à ce terminal et disparaît quand on le ferme.

2. Dans l'éditeur (PyCharm, VS Code). L'IDE possède son propre réglage de projet, le « Python Interpreter », qui lui indique quel `python.exe` utiliser pour exécuter, déboguer, autocompléter et lister les paquets. Ce réglage est stocké dans la configuration de l'IDE, pas dans le terminal.

Ces deux mécanismes ne communiquent pas. PyCharm ne lit pas l'état d'activation de ton terminal. D'où le paradoxe que tu observes si : ton terminal affiche bien `(.venv)` et `python` y fonctionne, mais PyCharm continue d'afficher « No interpreter selected » — parce que SON réglage à lui n'a jamais été renseigné. Les deux sont des pointeurs distincts vers un `python.exe` ; en régler un ne règle pas l'autre.

La solution est de renseigner l'interpréteur dans PyCharm une bonne fois : **Settings → Project → Python Interpreter → Add Interpreter → Existing**, puis sélectionner le `python.exe` de ton environnement.

```
# l'interpreteur a designer dans l'IDE = le python.exe de l'environnement
```

```
Windows      : <projet>\.venv\Scripts\python.exe
macOS/Linux : <projet>/.venv/bin/python
```

Une fois ce réglage fait, le message disparaît, l'autocomplétion connaît tes paquets, et le terminal intégré de PyCharm active même automatiquement l'environnement à l'ouverture. Le message qui revient malgré une activation signifie donc presque toujours : « l'IDE n'a pas (ou plus) de `python.exe` valide configuré » — souvent parce que l'environnement a été recréé ou déplacé.

Pourquoi ça compte. Comprendre que le terminal et l'IDE choisissent l'interpréteur séparément évite des heures de « *pourtant c'est activé !* ». En pratique, pour chaque projet, on fait deux gestes complémentaires : activer l'environnement dans le terminal, ET pointer l'interpréteur du projet dans l'IDE vers ce même `python.exe`.

3. Sans isolation : le problème des dépendances

Quand on installe Python, on obtient un environnement **global** : un interpréteur et un seul dossier de paquets (`site-packages`) partagé par tous les projets. Tant qu'on n'a qu'un projet, cela suffit. Mais dès qu'on en a deux, le conflit surgit : le projet A exige `pandas 1.5`, le projet B exige `pandas 2.2`. Un dossier partagé unique ne peut pas contenir les deux versions ; installer l'une casse l'autre. C'est l'**enfer des dépendances** (`dependency hell`).

```
Environnement GLOBAL (un seul site-packages partagé)
projet_A veut pandas 1.5  --+
projet_B veut pandas 2.2  ---> impossible : une seule version a la fois
```

La parade est universelle : donner à **chaque projet son propre environnement**, isolé, avec ses propres paquets. C'est ce que font, chacun à leur manière, les outils de la section suivante.

Pourquoi ça compte. Une firme exécute plusieurs stratégies, souvent figées sur des versions précises pour ne pas altérer les résultats. L'isolation garantit qu'ajuster une dépendance pour l'une n'aura jamais d'effet de bord sur les autres.

4. Les méthodes pour créer un environnement

Pour pouvoir comparer les outils, on réalise avec **chacun la même tâche**, en quatre étapes identiques : **(1) créer** l'environnement, **(2) l'activer**, **(3) installer pandas**, **(4) figer** le tout pour le reproduire. Seules les commandes changent ; l'objectif est rigoureusement le même.

4.1 venv (+ pip) — l'outil standard

`venv` est fourni avec Python : rien à installer. Il crée l'environnement à partir de l'interpréteur qui lance la commande (il n'installe donc pas une autre version de Python).

<code>python -m venv .venv</code>	# (1) créer
<code>.venv\Scripts\Activate.ps1</code>	# (2) activer (Windows PowerShell)
<code>source .venv/bin/activate</code>	# (2) activer (macOS / Linux)
<code>pip install pandas</code>	# (3) installer pandas
<code>pip freeze > requirements.txt</code>	# (4) figer les versions

4.2 virtualenv — l'ancêtre de venv

Quasiment identique à `venv`, mais c'est un paquet externe (à installer) dont `venv` s'inspire. Un peu plus rapide, compatible avec d'anciennes versions de Python, et capable de cibler un interpréteur déjà installé.

```

pip install virtualenv          # virtualenv n'est PAS fourni avec Python
virtualenv .venv               # (1) créer
.venv\Scripts\Activate.ps1    # (2) activer (identique à venv)
source .venv/bin/activate      # (2) activer (macOS / Linux)
pip install pandas             # (3) installer pandas
pip freeze > requirements.txt   # (4) figer

```

4.3 conda — environnements + paquets, même non-Python

conda gère à la fois l'environnement ET les paquets, y compris des bibliothèques système non-Python (C, Fortran). Il installe lui-même la version de Python demandée et crée des environnements **nommés**, indépendants d'un dossier projet.

```

conda create -n trading python=3.11 pandas  # (1)+(3) crée + installe Python 3.11
et pandas
conda activate trading                     # (2) activer (par son NOM)
conda install pandas                       # (3) (déjà fait ci-dessus, sinon
ainsi)
conda env export > environment.yml         # (4) figer

```

4.4 poetry — tout-en-un, déclaratif

poetry gère l'environnement virtuel **pour toi** et résout les dépendances. On déclare ce qu'on veut dans un `pyproject.toml` ; poetry écrit un `poetry.lock` qui fige tout l'arbre résolu.

```

pip install poetry                # à installer une fois
poetry init                      # (1) génère pyproject.toml (crée l'env au
besoin)
poetry add pandas                # (3) installe pandas ET écrit poetry.lock
poetry run python mon_script.py  # (2) exécuter DANS l'env (sans 'activer' à la
main)
# (4) figer : poetry.lock EST le verrouillage, à versionner avec le code

```

4.5 uv — pip + venv, en bien plus rapide

uv (écrit en Rust) réunit les rôles de pip et venv avec une vitesse remarquable, et peut même télécharger des versions de Python. C'est un remplaçant moderne, de plus en plus répandu.

```

pip install uv                   # ou installeur dédié
uv venv                         # (1) créer (très rapide)
.venv\Scripts\Activate.ps1      # (2) activer (ou s'en passer via 'uv run')
source .venv/bin/activate       # (2) activer (macOS / Linux)
uv pip install pandas           # (3) installer pandas
uv pip freeze > requirements.txt # (4) figer (ou, en mode projet : uv.lock)

```

Pourquoi ça compte. Le même besoin — un environnement isolé avec pandas, reproductible — se règle avec n'importe lequel de ces outils. Savoir les reconnaître et passer de l'un à l'autre, c'est pouvoir rejoindre n'importe quel projet existant sans le casser.

5. Comparaison des méthodes

Les cinq outils visent le même but mais ne se valent pas selon le contexte. Le tableau résume leurs différences ; les paragraphes qui suivent les opposent deux à deux.

Outil	Inclus avec Python	Gère la version de Python	Paquets non-Python	Fichier de repro	Idéal pour
venv	Oui	Non	Non	requirements.txt	Le cas standard ; réflexe par défaut
virtualenv	Non	Partiel*	Non	requirements.txt	Projets hérités, anciennes versions
conda	Non	Oui	Oui	environment.yml	Science des données, libs compilées
poetry	Non	Non	Non	poetry.lock	Applications structurées, repro stricte
uv	Non	Oui	Non	uv.lock	Pip/venv en bien plus rapide ; moderne

* virtualenv peut **cibler** une autre version de Python déjà installée, mais ne l'installe pas lui-même.

venv vs virtualenv : la même chose, à peu de chose près. venv est intégré à Python, virtualenv est externe mais légèrement plus rapide et sait viser un interpréteur déjà présent. Pour un nouveau projet, venv suffit.

venv/virtualenv vs conda : conda va plus loin en installant lui-même la version de Python et des bibliothèques **non-Python** compilées (utile pour des piles scientifiques lourdes). En contrepartie il est plus volumineux et raisonne en environnements nommés. On le choisit surtout en science des données.

venv+pip vs poetry : poetry ajoute la **résolution** des dépendances, un fichier de **verrouillage** et un pyproject.toml déclaratif, et gère l'environnement à ta place. Plus de structure et de garanties, mais davantage à apprendre que le simple venv + pip.

uv vs tous les autres : uv fait ce que font pip et venv mais beaucoup plus vite, sait gérer les versions de Python et produire un verrouillage. C'est le candidat moderne quand la vitesse et la rigueur comptent.

***Pourquoi ça compte.** Le bon outil dépend du projet, pas d'une mode : venv pour l'immense majorité des cas, conda pour le scientifique lourd, poetry/uv pour une application structurée. Savoir pourquoi on choisit l'un plutôt qu'un autre est plus utile que de tous les maîtriser.*

6. Reproduire un environnement : les fichiers

Un environnement n'a de valeur que s'il est reproductible. Chaque méthode a son fichier, mais tous séparent deux idées : **ce que le projet veut** et **ce qui a été réellement installé**.

requirements.txt (venv, virtualenv, uv) : une liste de paquets avec versions épinglées, produite par pip freeze et réinstallée par pip install -r requirements.txt. Simple et universel.

```
pandas==2.2.0
numpy==1.26.4
```

pyproject.toml (poetry, uv) : déclaratif. On y note les dépendances **directes** voulues (par ex. pandas), sans figer tout le détail. Il dit l'intention.

Fichier de verrouillage — poetry.lock, uv.lock : il fige l'arbre **complet** résolu (versions exactes, y compris les dépendances des dépendances). C'est lui qui garantit une reproductibilité parfaite, là où un requirements.txt écrit à la main peut être incomplet. **environment.yml** joue ce rôle côté conda.

La règle : le pyproject.toml décrit **ce que je veux** ; le lock fige **ce qui a été résolu et installé**. On versionne ces fichiers avec le code (jamais le dossier .venv lui-même, qui se recrée).

Pourquoi ça compte. Reproduire un backtest à l'identique exige de figer non seulement pandas, mais toute la chaîne de dépendances. C'est le rôle du fichier de verrouillage : sans lui, une dépendance indirecte mise à jour peut suffire à changer un résultat financier.

7. La démarche recommandée

La gestion des environnements est une habitude à appliquer dès le premier jour d'un projet, pas un correctif de dernière minute.

1. créer un environnement DEDIE au projet (venv par défaut)
2. l'activer dans le terminal (.venv\Scripts\Activate.ps1 ...)
3. pointer l'interpreteur de l'IDE vers ce meme python.exe
4. installer les paquets DANS l'environnement (pip install ...)
5. figer pour reproduire (pip freeze > requirements.txt)
6. ne JAMAIS installer en global pour un projet ; versionner le fichier de repro

Quel outil ? venv + pip pour la grande majorité des cas ; conda si le projet dépend de piles scientifiques ou de bibliothèques non-Python ; poetry ou uv pour une application plus structurée avec verrouillage. Le principe ne change pas : **un environnement isolé par projet, des versions figées, un IDE pointé sur le bon interpréteur.**

Pourquoi ça compte. Adopter ce réflexe dès le départ supprime l'essentiel des problèmes « ça marche chez moi mais pas en production » — exactement l'hygiène attendue là où un code doit tourner de façon identique du poste de développement au serveur de trading.

8. Le terminal qui s'active automatiquement (ou non)

Tu as sans doute remarqué qu'à l'ouverture d'un projet, le terminal intégré affiche parfois d'emblée le préfixe (.venv) — l'environnement est déjà activé — et que d'autres fois rien ne se passe. Ce n'est pas aléatoire : cela dépend entièrement de ce que l'IDE sait de ton projet.

Quand l'activation est automatique. L'éditeur active le bon environnement à l'ouverture d'un nouveau terminal si deux conditions sont réunies : (1) un interpréteur de projet est bien renseigné (cf. §2), et (2) l'option correspondante est active — dans PyCharm, Settings → Tools → Terminal → « Activate virtualenv » ; dans VS Code, python.terminal.activateEnvironment (activée par défaut). L'IDE injecte alors la commande d'activation dans chaque nouveau terminal.

Quand elle ne se fait pas. Si aucun interpréteur n'est configuré, si l'environnement a été recréé ou déplacé (le python.exe pointé n'existe plus), ou si tu ouvres un terminal système extérieur à l'IDE, personne n'active quoi que ce soit : tu retombes sur le Python global. Un terminal système n'a aucune connaissance du projet — c'est à toi d'y lancer activate.

Verifier QUEL python est reellement actif dans un terminal

```
python -c "import sys; print(sys.executable)"
```

```
# -> ...\.venv\Scripts\python.exe => environnement bien actif
```

```
# -> C:\Python311\python.exe => Python global : activation manquante
```

Pourquoi ça compte. Avant de lancer une stratégie, ce réflexe (afficher sys.executable) lève le doute en une seconde : on sait si le code tournera avec les versions figées du projet ou avec un Python global aux dépendances imprévisibles.

9. Exécuter un script : le bouton RUN et les configurations d'exécution

Lancer un script peut se faire de deux manières, et il est utile de comprendre qu'elles sont équivalentes.

En ligne de commande, on écrit python mon_script.py dans un terminal activé. Dans l'IDE, le bouton RUN (▶) fait exactement la même chose — mais il mémorise les réglages dans une configuration d'exécution (run configuration).

Une configuration d'exécution est une fiche nommée qui décrit comment lancer un programme. Elle regroupe l'interpréteur à utiliser (le python.exe du projet), le script ou le module cible, les arguments passés au programme (comme sur la ligne de commande), le répertoire de travail (working directory, d'où les chemins relatifs sont résolus) et les variables d'environnement (par exemple une clé d'API, ou un mode PAPER vs LIVE).

```
# Ce que reproduit une configuration d'exécution, en ligne de commande :
cd C:\projets\trading                # working directory
set API_ENV=PAPER                    # variable d'environnement (Windows)
.venv\Scripts\python.exe backtest.py --symbole ES --jours 30  # interpreteur +
script + args
```

L'intérêt est la reproductibilité et le confort : au lieu de retaper la commande, on garde plusieurs configurations prêtes — par exemple « Backtest ES », « Backtest NQ », « Live (PAPER) » — et on bascule de l'une à l'autre d'un clic. Dans PyCharm : Run → Edit Configurations... ; dans VS Code, elles se décrivent dans .vscode/launch.json.

Pourquoi ça compte. Un même script de trading se lance dans des contextes très différents (symbole, période, compte réel ou simulé). Nommer ces contextes dans des configurations évite l'erreur classique — lancer en réel une commande destinée au bac à sable — et rend chaque exécution rejouable à l'identique.

10. Choisir son éditeur : PyCharm, VS Code, Visual Studio 2026

Trois éditeurs reviennent souvent ; ils ne visent pas le même usage. Le tableau les compare, puis on résume comment choisir.

Éditeur	Type	Points forts	Limites	Idéal pour
PyCharm	IDE Python dédié	Outillage Python très complet (débugueur, refactoring, gestion d'environnements) ; « tout marche » sans configuration	Lourd, plus lent à démarrer ; édition Pro payante	Gros projets Python, confort maximal
VS Code	Éditeur extensible	Léger et rapide ; polyvalent (multi-langages) via extensions (Python, Pylance) ; gratuit	Dépend d'extensions à configurer ; moins « clé en main » pour du Python pur	Usage général, projets mixtes, machines modestes
Visual Studio 2026	IDE complet (Microsoft)	Très puissant pour l'écosystème .NET/C++ ; supporte aussi Python	Surdimensionné pour du Python seul ; orienté Windows/.NET	Projets .NET, ou Python intégré à une solution .NET

Comment choisir. Pour du Python pur, l'arbitrage se joue surtout entre PyCharm (le plus complet et guidé) et VS Code (le plus léger et polyvalent) — c'est largement une question de préférence. Visual Studio 2026 ne s'impose que si tu travailles déjà dans l'univers .NET/C++ et veux y intégrer du Python ; à ne pas confondre avec VS Code, produit distinct malgré le nom voisin.

Pourquoi ça compte. Sur un poste de trading, l'éditeur n'exécute pas la stratégie — l'interpréteur le fait (§2). Le choix de l'IDE relève donc du confort et de la vitesse de travail, pas de la performance du code : inutile d'en changer par mode.

11. SDK, runtime, écosystèmes : Python vs .NET

Le mot SDK (Software Development Kit) revient dans d'autres écosystèmes ; le comprendre par comparaison clarifie ce qu'est, au fond, « installer Python ». Trois notions sont à distinguer : le runtime, ce qui exécute le programme (en Python, l'interpréteur `python.exe` ; en .NET, le CLR) ; le SDK, le runtime plus les outils de développement (compilateur, outils de build, bibliothèque standard) — installer Python fournit déjà l'équivalent d'un petit SDK (interpréteur + `pip` + `stdlib`) ; et l'écosystème de paquets, où l'on récupère les bibliothèques tierces (PyPI via `pip` pour Python ; NuGet pour .NET).

Aspect	Python	.NET (C#)
Exécution	Interprété (bytecode exécuté par l'interpréteur)	Compilé en IL, exécuté par le CLR (JIT)
« Installation de base »	Interpréteur + <code>pip</code> + <code>stdlib</code>	SDK .NET (runtime + compilateur + outils)
Paquets tiers	PyPI (<code>pip</code>)	NuGet
Typage	Dynamique (annotations optionnelles, cf. module 3 §6)	Statique, vérifié à la compilation
Vitesse brute	Plus lente ; on vectorise/compile (cf. modules 5 et 8)	Généralement plus rapide (compilé)
Terrain de prédilection	Recherche, data science, prototypage rapide	Applications d'entreprise, typage fort

En trading, cette distinction n'est pas anecdotique : Python domine la recherche et le prototypage (rapidité d'écriture, écosystème scientifique), tandis que des composants à très faible latence sont parfois écrits en langage compilé (.NET, C++). Les deux mondes cohabitent souvent : prototype en Python, brique critique en langage compilé.

Pourquoi ça compte. Savoir que « Python », « SDK » et « runtime » désignent des couches différentes évite bien des confusions d'installation — et comprendre les forces respectives de Python et de .NET aide à choisir le bon outil pour chaque brique d'une chaîne de trading, au lieu de tout écrire dans un seul langage par habitude.

Récapitulatif

- 1. Environnement** : une version de Python + ses paquets ; il répond à deux besoins, l'isolation et la reproductibilité.
- 2. Interpréteur** : un `python.exe` précis ; le terminal (`activate`) et l'IDE (réglage « Python Interpreter ») le choisissent **séparément** — d'où le message de PyCharm malgré un terminal activé.
- 3. Sans isolation** : un `site-packages` global partagé → conflits de versions entre projets (enfer des dépendances).
- 4. Méthodes** : `venv`, `virtualenv`, `conda`, `poetry`, `uv` — la même tâche (créer, activer, installer, figer) avec des commandes différentes.
- 5. Comparaison** : `venv` (standard), `virtualenv` (hérité), `conda` (non-Python), `poetry` (structuré + lock), `uv` (rapide + lock).
- 6. Reproduire** : `requirements.txt` (figé), `pyproject.toml` (déclaré), `lock` / `environment.yml` (arbre complet figé).

7. Terminal auto-activ   : l'IDE active l'environnement au nouveau terminal si un interpr  teur de projet est renseign   et l'option activ  e ; sinon (env recr    /d  plac  , terminal syst  me), on retombe sur le Python global.
8. Ex  cution : le bouton RUN   quivaut    python script.py ; une configuration d'ex  cution m  morise interpr  teur, script, arguments, r  pertoire de travail et variables d'environnement.
9.   diteurs : PyCharm (complet, guid  ) et VS Code (l  ger, polyvalent) pour Python ; Visual Studio 2026 surtout pour l'  cosyst  me .NET/C++.
10. SDK / runtime /   cosyst  me : runtime = ce qui ex  cute (python.exe / CLR) ; SDK = runtime + outils ; paquets via PyPI (Python) ou NuGet (.NET).

Questions de v  rification

Essaie de r  pondre avant de lire la r  ponse, fournie juste en dessous, en retrait.

Sur l'environnement et l'interpr  teur

Q1. Qu'est-ce qu'un environnement, et    quels deux besoins r  pond-il ?

R  ponse. C'est une version de Python associ  e    un ensemble de paquets    des versions pr  cises. Il r  pond    l'isolation (les projets ne se g  nent pas) et    la reproductibilit   (recr  er le m  me contexte ailleurs).

Q2. Pourquoi PyCharm peut-il afficher « aucun interpr  teur » alors que le terminal est bien activ   ?

R  ponse. Parce que le terminal et l'IDE choisissent l'interpr  teur s  par  ment. L'activation ne concerne que la fen  tre de terminal ; PyCharm, lui, a son propre r  glage « Python Interpreter » qui n'a pas   t   renseign  . En r  gler un ne r  gle pas l'autre.

Q3. Comment faire dispara  tre ce message dans PyCharm ?

R  ponse. Renseigner l'interpr  teur du projet : Settings → Project → Python Interpreter → Add → Existing, puis s  lectionner le python.exe de l'environnement (.venv\Scripts\python.exe sous Windows, .venv/bin/python sous macOS/Linux).

Sur le probl  me des d  pendances

Q4. Pourquoi l'environnement global pose-t-il probl  me avec plusieurs projets ?

R  ponse. Il n'a qu'un seul site-packages partag   : deux projets exigeant des versions diff  rentes d'un m  me paquet ne peuvent pas coexister. Installer l'une casse l'autre — l'enfer des d  pendances.

Q5. Quelle est la parade universelle ?

R  ponse. Donner    chaque projet son propre environnement isol  , avec ses propres paquets, pour qu'une modification dans l'un n'affecte jamais les autres.

Sur les m  thodes et leur comparaison

Q6. Quelles quatre   tapes sont communes    toutes les m  thodes ?

R  ponse. Cr  er l'environnement, l'activer, installer les paquets, puis figer le tout pour le reproduire. Seules les commandes diff  rent d'un outil    l'autre.

Q7. Qu'apporte conda que venv n'offre pas ?

Réponse. conda installe lui-même la version de Python demandée et des bibliothèques non-Python compilées (C, Fortran), et raisonne en environnements nommés. venv se limite à des paquets Python à partir de l'interpréteur courant.

Q8. Qu'ajoute poetry par rapport à venv + pip ?

Réponse. La résolution des dépendances, un pyproject.toml déclaratif et un poetry.lock qui fige tout l'arbre résolu, le tout en gérant l'environnement à ta place — plus de garanties, mais plus à apprendre.

Q9. Différence entre requirements.txt / pyproject.toml et un fichier de verrouillage ?

Réponse. requirements.txt et pyproject.toml expriment ce que le projet veut (dépendances directes) ; le lock (poetry.lock, uv.lock, environment.yml) fige l'arbre complet résolu, garantissant une reproductibilité parfaite.

Sur l'outillage : terminal, exécution et IDE

Q10. Pourquoi le terminal s'active-t-il automatiquement dans le bon environnement à l'ouverture d'un projet, et pourquoi parfois non ?

Réponse. Parce que l'IDE injecte la commande d'activation dans chaque nouveau terminal quand un interpréteur de projet est renseigné et l'option d'activation active. Cela ne se produit pas si aucun interpréteur n'est configuré, si l'environnement a été recréé ou déplacé, ou dans un terminal système extérieur à l'IDE — on utilise alors le Python global.

Q11. Que mémorise une configuration d'exécution, et à quoi sert-elle ?

Réponse. L'interpréteur, le script cible, les arguments, le répertoire de travail et les variables d'environnement. Elle rend chaque lancement reproductible et permet de basculer d'un contexte à l'autre (ex. Backtest ES, Live PAPER) sans retaper la commande.

Q12. Qu'est-ce qu'un SDK par rapport à un runtime, et où se situe « installer Python » ?

Réponse. Le runtime exécute le programme (python.exe, ou le CLR côté .NET) ; le SDK ajoute les outils de développement (compilateur, build, bibliothèque standard). Installer Python fournit déjà l'équivalent d'un petit SDK : interpréteur + pip + bibliothèque standard.

Ce module complète la série : maîtriser les environnements — et savoir relier le bon interpréteur au terminal comme à l'éditeur — c'est garantir qu'un code qui fonctionne sur le poste de développement fonctionnera aussi en production, à l'identique.